

DOBLE-M SOFTWARE

Dirección y Gestión de Proyectos

Minerva Cebrián Marín (mcm17@correo.ugr.es)

Cristina García Gázquez (cristi05@correo.ugr.es)

Ángela García Olmos (angelagrc@correo.ugr.es)

Kirsten Gazy Manalo (kirstenmanalo@correo.ugr.es)

Pedro Javier Pérez Jiménez (pjpz@correo.ugr.es)

Daniel Vega Jiménez (deltac@correo.ugr.es)

Curso 2026-2027



UNIVERSIDAD
DE GRANADA

Índice

1. Cronograma **2**
1.1. Vista en Tabla . . . 2
1.2. Vista en Diagrama de Gantt . . . 3

2. Documentos generados durante la Primera Iteración **4**

3. Bocetos **4**
3.1. Bocetos Agenda . . . 4
3.2. Bocetos Base de Datos . . . 21

4. Manual de Calidad **22**
4.1. Objeto y Alcance del Manual de Calidad . . . 22
4.2. Criterios y Dimensiones Objetivas de Calidad . . . 22
4.2.1. Dimensiones de Calidad para la Documentación (LaTeX) . . . 22
4.2.2. Dimensiones de Calidad para el Código Fuente . . . 22
4.2.3. Dimensiones de Calidad en Accesibilidad Cognitiva y Usabilidad 23
4.3. Procedimientos de Control de Calidad durante el Desarrollo . . . 23
4.4. Medición del Desempeño y Control de Calidad Interno . . . 24

5. Manual de Despliegue, Arquitectura y Especificación de la API **25**
5.1. Preparación del Entorno y Despliegue con Docker . . . 25
5.1.1. Inicialización por primera vez (Instalación limpia) . . . 25
5.1.2. Recompilación del entorno (Entorno previamente montado) . . 25
5.1.3. Instalación del motor de plantillas Twig . . . 25
5.1.4. Despliegue e Importación de la Base de Datos . . . 26
5.2. Arquitectura y Estructura del Proyecto . . . 27
5.3. Especificación de la API HTTP / JSON . . . 28
5.3.1. Estándar de Respuesta JSON . . . 28
5.3.2. Catálogo de Endpoints Disponibles . . . 28
5.3.3. Diccionario de Códigos de Respuesta . . . 30

1 Cronograma

Para garantizar un seguimiento riguroso del desarrollo de la Segunda Iteración (comprendida entre el 12 de noviembre y el 2 de diciembre de 2026), el equipo ha estructurado la distribución temporal de tareas diferenciando los bloques de trabajo del *Front-End* y del *Back-End*.

Dado el reajuste de alcance aplicado tras la primera iteración, la planificación actual se focaliza en consolidar las funcionalidades de inicio de sesión de familias, selección de alumnos, marcado de progreso en actividades y la asignación de relaciones en el panel de administración.

A continuación, se presenta esta planificación detallada en dos formatos complementarios: en primer lugar, el desglose cronológico por fases, entregables e hitos en formato tabular; y en segundo lugar, la representación visual del calendario mediante un Diagrama de Gantt.

1.1. Vista en Tabla

Fase	**Tipo**	**Fecha**	**Principales resultados**
Frontend	Desarrollo	12/11 – 16/11	Implementar inicios de sesión.
Frontend	Desarrollo	17/11 – 19/11	Implementar interfaz de crear actividad y asignar.
Frontend	Desarrollo	20/11 – 21/11	Implementar vista de actividad.
Frontend	Desarrollo	22/11 – 25/11	Implementar interfaz de agenda.
Frontend	Desarrollo	26/11	Implementar interfaz de creación de usuarios y grupos.
Frontend	Desarrollo	27/11	Implementar interfaz de asignación estudiante-profesor y grupos.
Frontend	Desarrollo	28/11	Implementar interfaz de asignación estudiante-familiar.
Frontend	Integración	29/11 – 02/12	Integración Frontend-Backend.
Backend	API	12/11 – 13/11	API de login familiar.
Backend	API	14/11 – 15/11	API de estudiantes asociados a familiares.
Backend	API	16/11 – 19/11	API de completación de una actividad y sus pasos. Revisión de BD.
Backend	Soporte	20/11 – 22/11	Apoyo al Frontend.
Backend	QA	23/11 – 24/11	Testeo de funcionalidades.
Backend	Integración	25/11 – 26/11	Integración Frontend-Backend.
Backend	Soporte	27/11 – 28/11	Apoyo al Frontend.
Backend	Integración	29/11 – 30/11	Integración Frontend-Backend.
Backend	QA	01/12 – 02/12	Testeo de funcionalidades.

Cuadro 1: Cronograma en formato tabla (Iteración 2).

1.2. Vista en Diagrama de Gantt

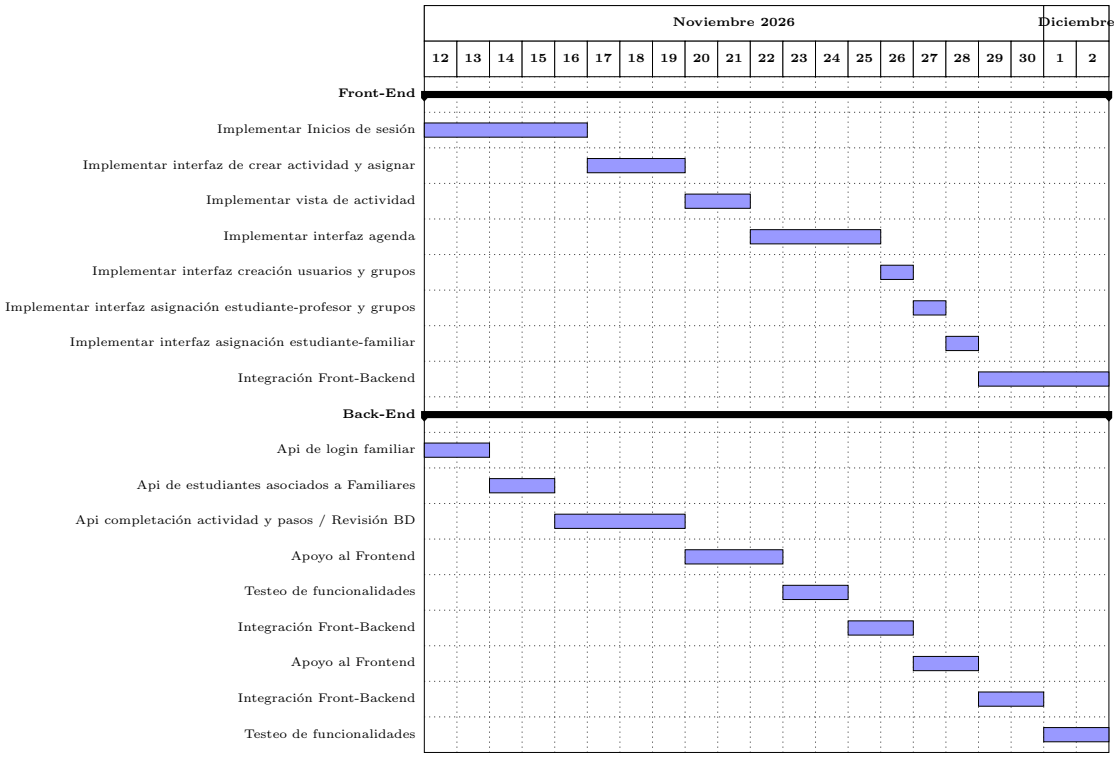


Figura 1: Diagrama de Gantt — Planificación detallada de la Iteración 2.

2 Documentos generados durante la Primera Iteración

3 Bocetos

Para realizar los bocetes se ha utilizado la herramienta Figma, que permite crear prototipos interactivos de forma sencilla y rápida. Los bocetos se han diseñado para reflejar la estructura y el flujo de navegación de la aplicación, así como para mostrar las principales funcionalidades y pantallas que se implementarán en el proyecto, para ver con mayor claridad se pueden usar el enlace :
https://www.figma.com/design/aNuYr9skZX7z6x5FqHQXfw/Trabajo-DGP?node-id=0-1&t=0Jn5VwT3RTUfXOOC-1

3.1. Bocetos Agenda

Para ver de forma clara la estructura de la aplicación y cómo se conectan las distintas pantallas, en primer lugar, se incluye el esquema general de navegación y, a continuación, se explica en detalle cada uno de los flujos de la interfaz.

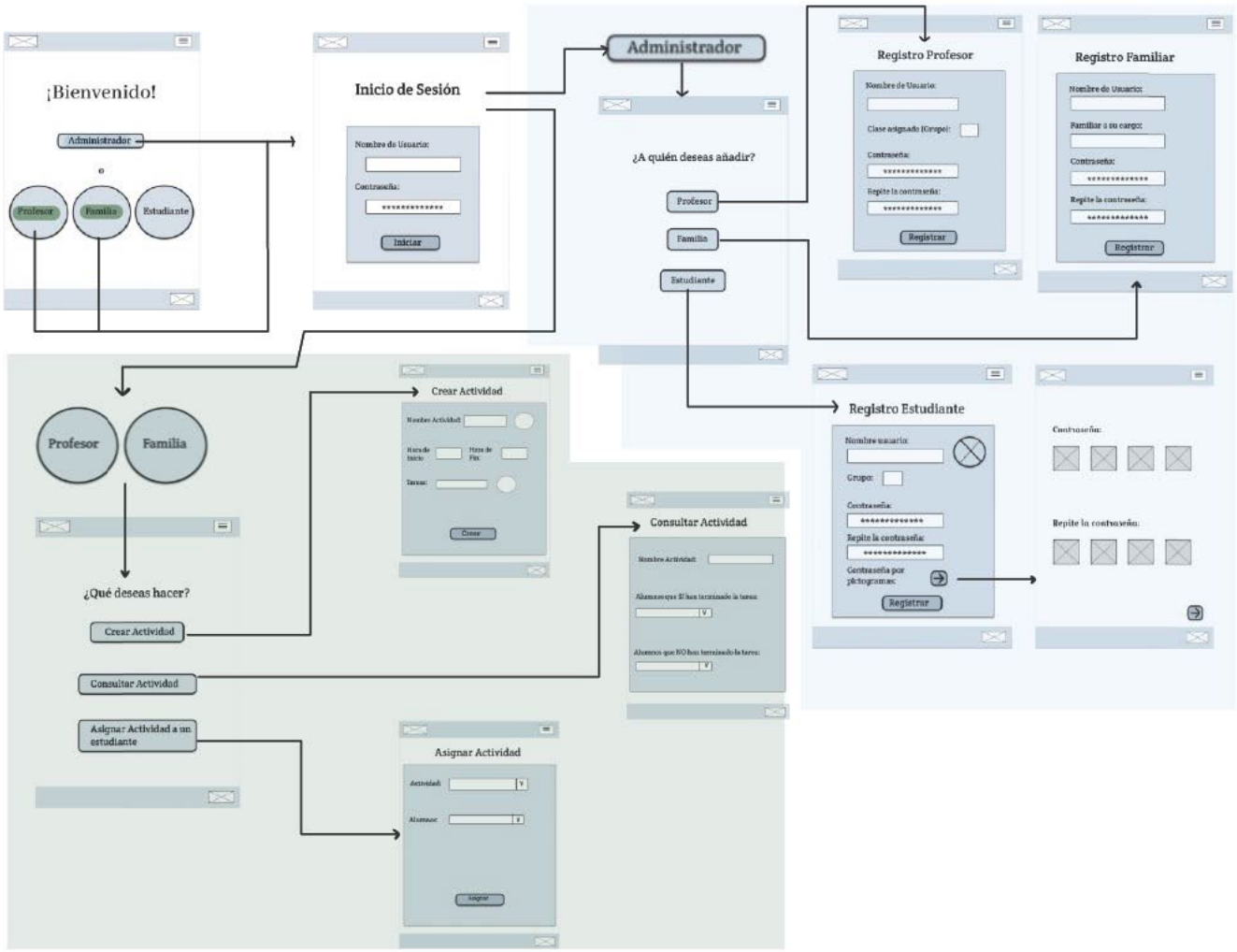


Figura 2: Esquema general de navegación de la aplicación.

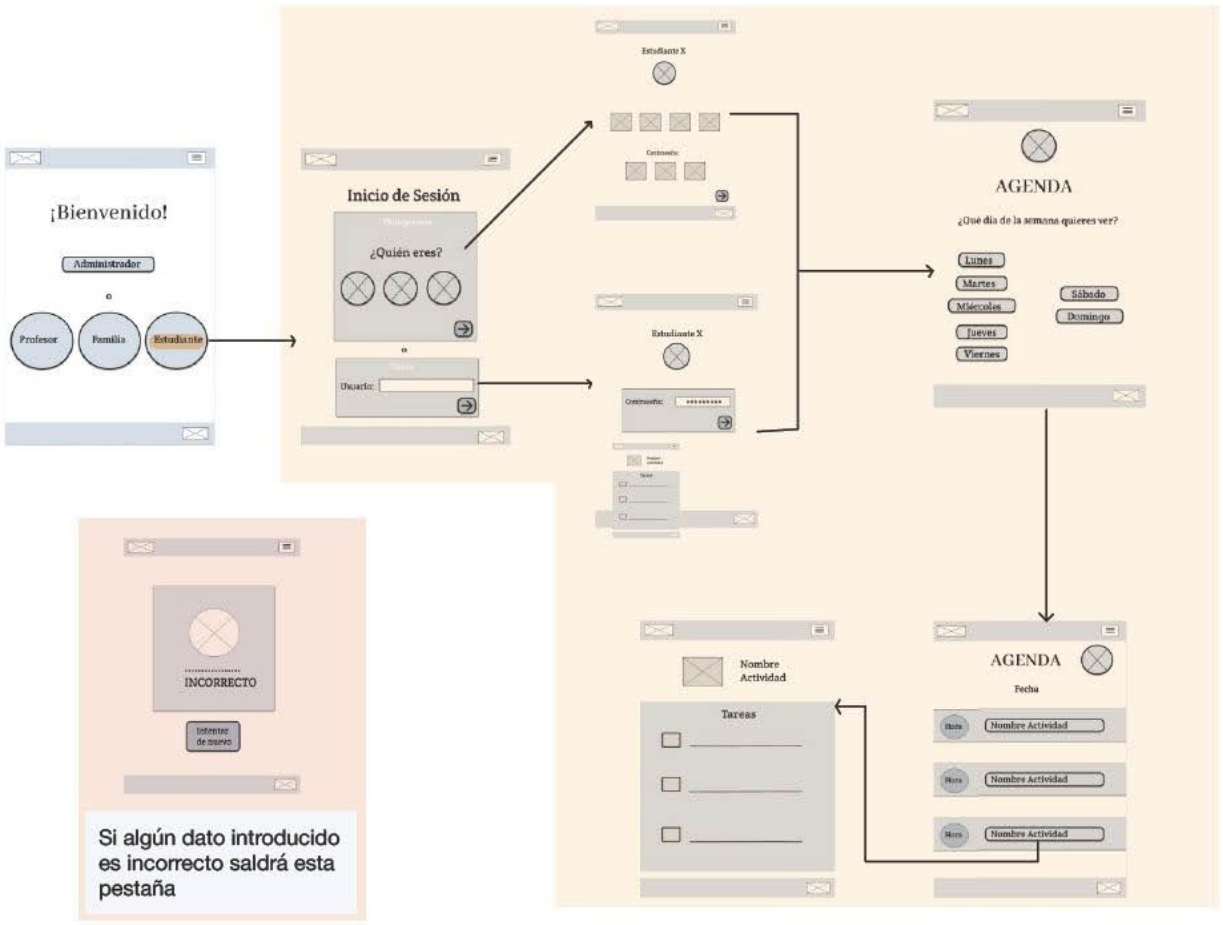


Figura 3: Esquema general de navegación de la aplicación.

En primer lugar, nos encontramos con una pantalla de bienvenida donde debemos elegir el perfil con el que deseamos acceder a la aplicación: Administrador, Profesor, Familia o Estudiante. Dependiendo de la opción escogida, el sistema redirige al usuario a su respectivo inicio de sesión o panel de control.



Figura 4: Pantalla de bienvenida y selección de perfil.

Si seleccionamos los perfiles de Administrador, Familiar o Profesor, la aplicación redirige a una interfaz de inicio de sesión unificada. En esta pantalla se solicita introducir el Nombre de Usuario y la Contraseña, con un botón para entrar a la aplicación.



Figura 5: Interfaz de inicio de sesión unificada para Administrador, Profesor y Familia.

Por otro lado, si seleccionamos el inicio de sesión del estudiante, este ofrece dos alternativas adaptadas a las necesidades del usuario, ya que se puede identificar mediante pictogramas («¿Quién eres?») o introduciendo el nombre de usuario por texto. Dependiendo de la opción escogida, la aplicación redirige a una pantalla u otra para introducir la contraseña: si se entra por pictogramas, la clave se introduce también de forma visual mediante una secuencia de iconos y, si se accede por texto, se utiliza un campo de contraseña tradicional.

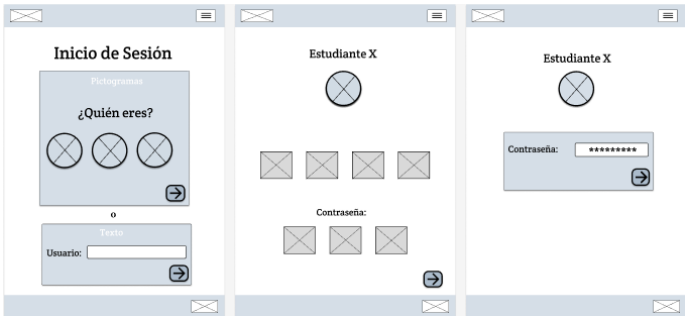


Figura 6: Opciones de inicio de sesión para el perfil de Estudiante.

Si iniciamos la sesión como Administrador, la interfaz muestra una pantalla con la pregunta «¿A quién deseas añadir?», permitiendo escoger entre tres opciones: Profesor, Familia o Estudiante. Dependiendo de la selección, el sistema redirige al usuario hacia el formulario de registro correspondiente para dar de alta al nuevo usuario en la plataforma.

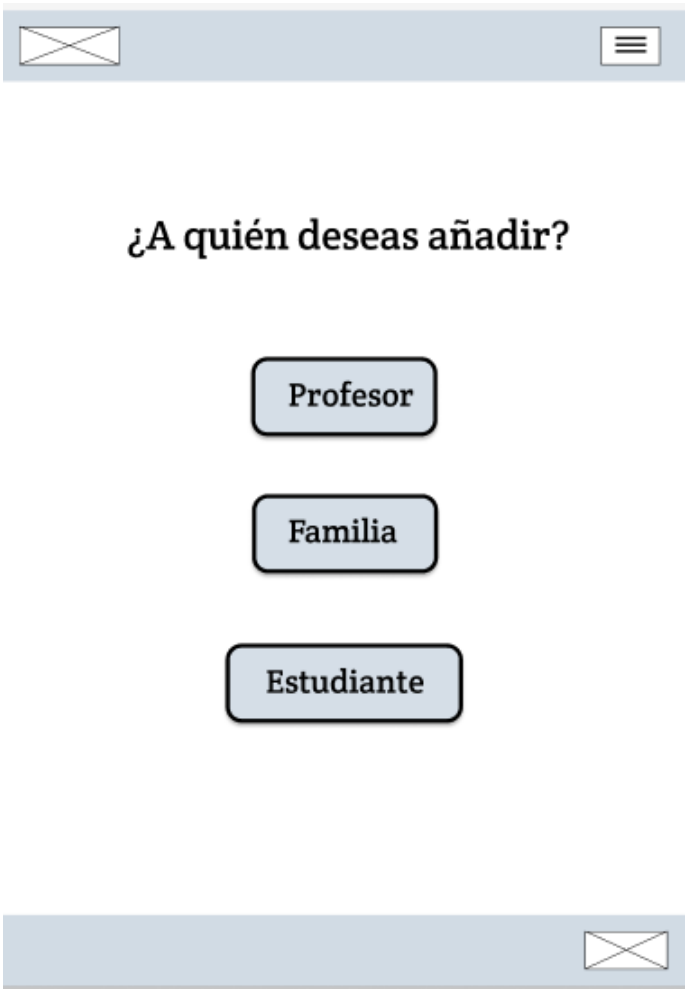


Figura 7: Interfaz de administración para el perfil de Administrador.

Si optamos por registrar un Profesor, la pantalla muestra un formulario donde se solicita el nombre de usuario, la clase asignada, la contraseña y su confirmación. De forma similar, si elegimos la opción de Familiar, el formulario solicita el nombre de usuario, el estudiante vinculado al familiar, la contraseña y su correspondiente repetición para validar el registro.



Figura 8: Interfaz de registro de usuarios para los perfiles de Profesor y Familiar.

Por otro lado, si elegimos la opción de Estudiante, accedemos a una pantalla de registro donde se solicita el nombre de usuario, el grupo y la contraseña mediante texto. No obstante, para aquellos estudiantes que se comunican o interactúan mejor de forma visual o que lo soliciten, la aplicación incluye la posibilidad de introducir y configurar la contraseña mediante pictogramas, donde de los cuatro pictogramas que se presentan, deberán seleccionar tres (pudiendo ser tres diferentes, tres iguales, dos iguales o la combinación que prefiera para su contraseña).

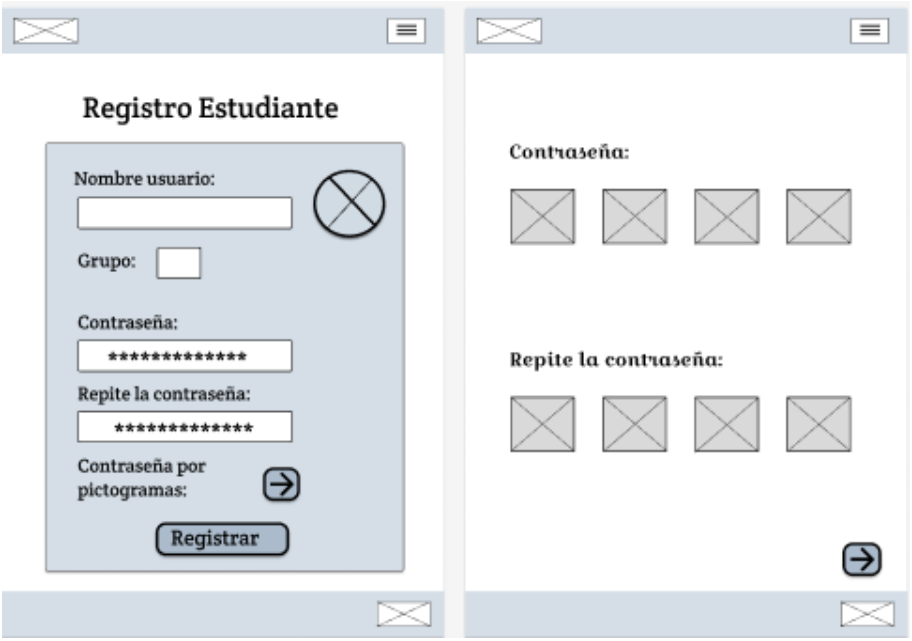


Figura 9: Interfaz de registro de usuarios para el perfil de Estudiante.

Si iniciamos sesión como Profesor o Familiar, la aplicación muestra una pantalla con la pregunta «¿Qué deseas hacer?». Desde aquí se puede elegir entre tres opciones principales mediante botones: «Crear Actividad», «Consultar Actividad» o «Asignar Actividad a un Estudiante».

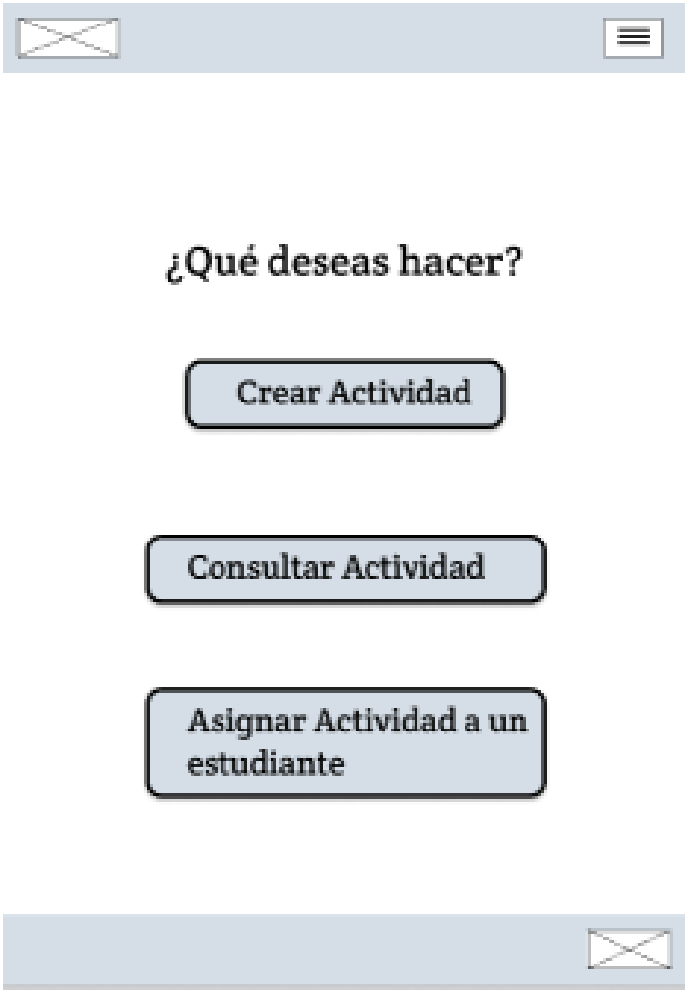


Figura 10: Pantallas de gestión de actividades para Profesor y Familia.

Si seleccionamos la opción de Crear Actividad, se despliega un formulario que permite configurar los detalles de la nueva actividad, donde se debe indicar el Nombre de la Actividad y el pictograma asociado, la Hora de Inicio y la Hora de Fin, así como un apartado para añadir las tareas vinculadas a la actividad y un botón de «Crear» para guardar los cambios.

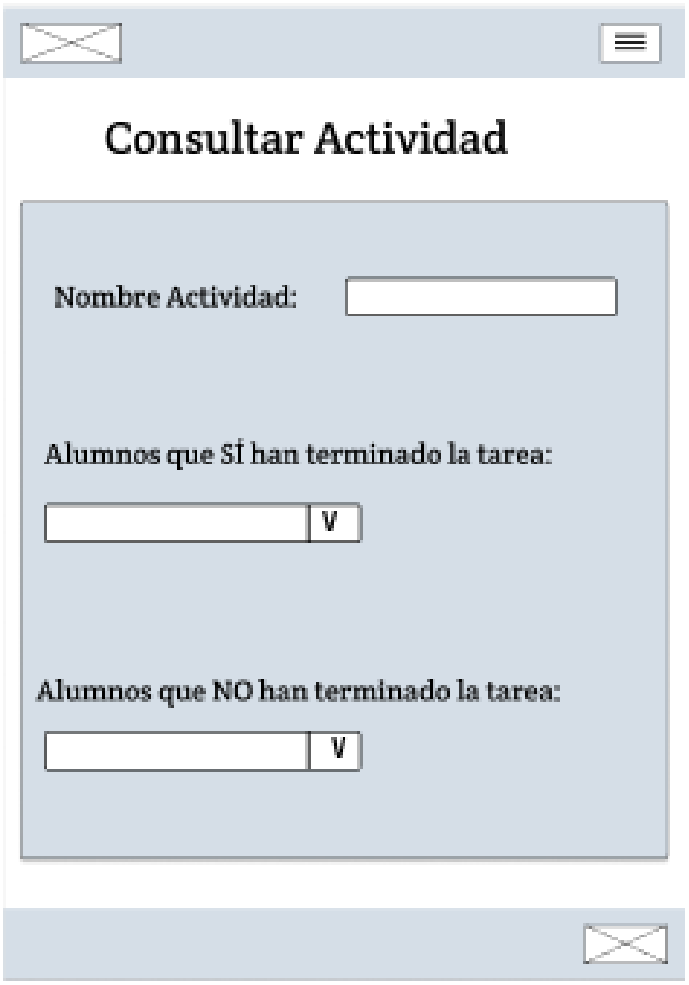


Figura 11: Pantallas de creación de actividades.

Por otro lado, si seleccionamos Consultar Actividad, accedemos a una pantalla de seguimiento que incluye el Nombre de la Actividad, las personas que la han completado y las personas que no. En el caso de los perfiles de Familiar, los listados desplegables de los alumnos que sí han terminado la tarea y de los alumnos que no han terminado la tarea se filtran de forma exclusiva para mostrar únicamente al alumno que tienen asociado a su cargo.

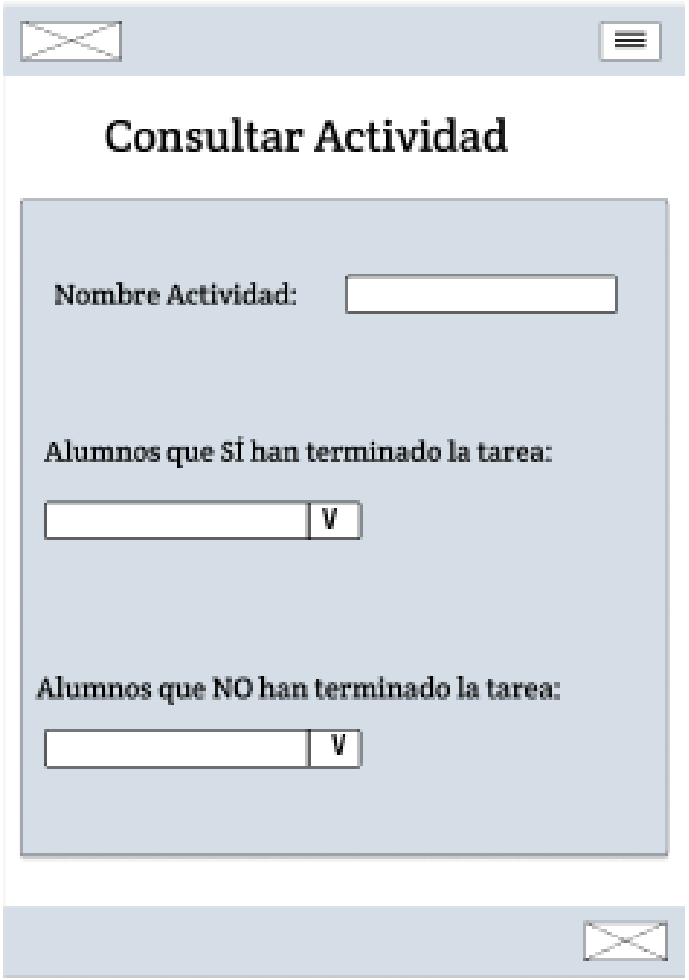


Figura 12: Pantallas de consulta de actividades.

Por último, si seleccionamos Asignar Actividad a un estudiante, tenemos dos campos desplegables: uno para seleccionar la Actividad y otro para elegir el Alumno o Alumnos al que se le va a asignar, finalizando con un botón de «Asignar». De igual forma que en «Consultar Actividad», si el usuario que realiza la acción es un Familiar, el desplegable de alumnos se restringe de forma que solo muestra al alumno que tiene a su cargo.

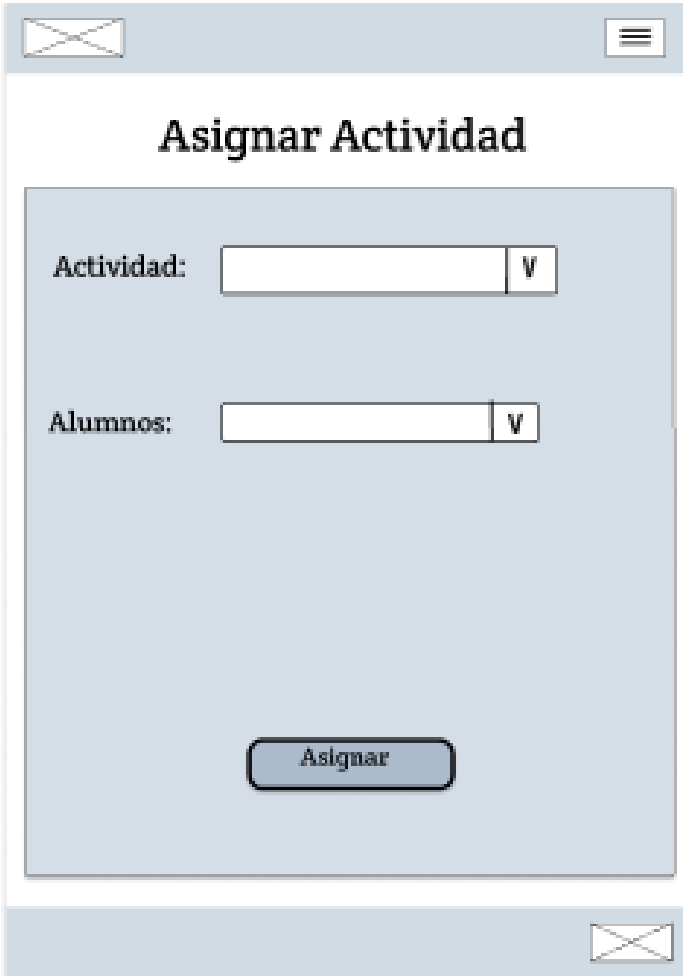


Figura 13: Pantallas de asignación de actividades.

Si iniciamos sesión como estudiante, se muestra la pantalla principal de su Agenda, donde se encuentra su imagen de usuario en la parte superior. Bajo el título, se plantea la pregunta «¿Qué día de la semana quieres ver?» y se presentan botones para seleccionar cada uno de los días.

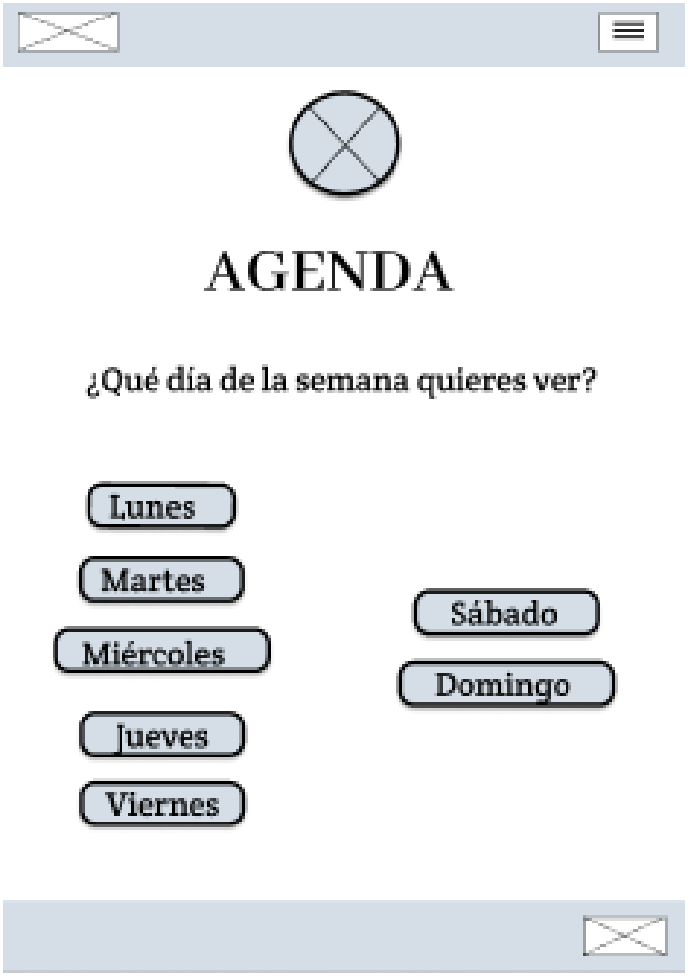


Figura 14: Pantalla principal de la Agenda del Estudiante.

Al pulsar sobre cualquiera de los días de la semana, la aplicación muestra la vista detallada de la Agenda para esa fecha concreta. En esta pantalla se despliega un listado con las actividades programadas, mostrando para cada una de ellas un círculo con la Hora de realización y un cuadro con el Nombre de la Actividad (que podrán ponerse en formato texto o con pictogramas).

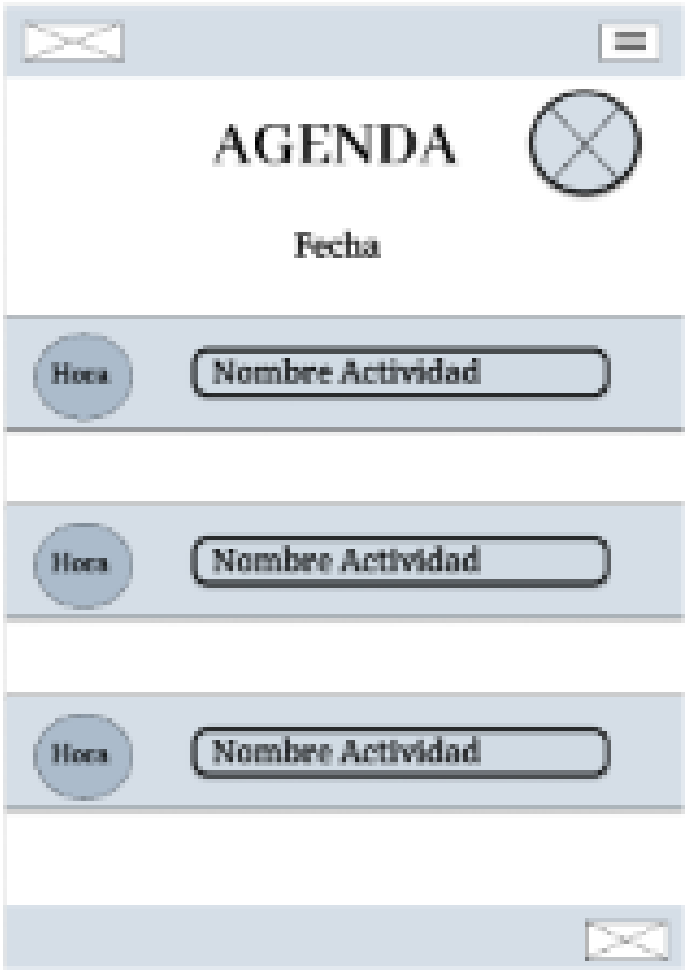


Figura 15: Pantalla principal, diaria y de detalle de tareas para el perfil Estudiante.

Al pulsar sobre una actividad específica, se accede a su pantalla de detalle. En la cabecera se muestra el Nombre de la Actividad acompañada de su pictograma asociado. Debajo, en la sección de Tareas, se despliega el listado donde cada elemento puede presentarse mediante texto o pictogramas dependiendo de la configuración del estudiante, incluyendo una casilla cuadrada a la izquierda diseñada para que el alumno pueda marcar la tarea como completada.

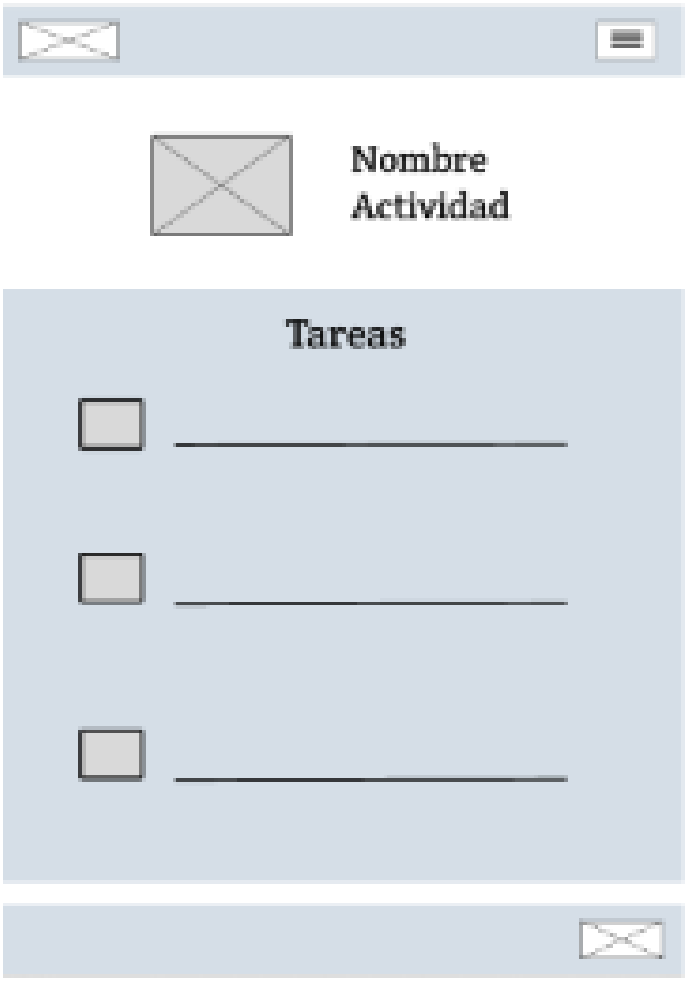


Figura 16: Vista de detalle de una actividad en la Agenda del Estudiante.

Para finalizar, la aplicación cuenta con una pantalla de error. En caso de que se introduzca un campo incorrecto en cualquiera de los formularios o procesos (como la contraseña o el nombre de usuario), el sistema redirige a esta interfaz, donde se muestra un icono superior, el texto correspondiente indicando qué campo ha sido incorrecto y un botón de «Intentar de nuevo» para volver a la pantalla anterior.

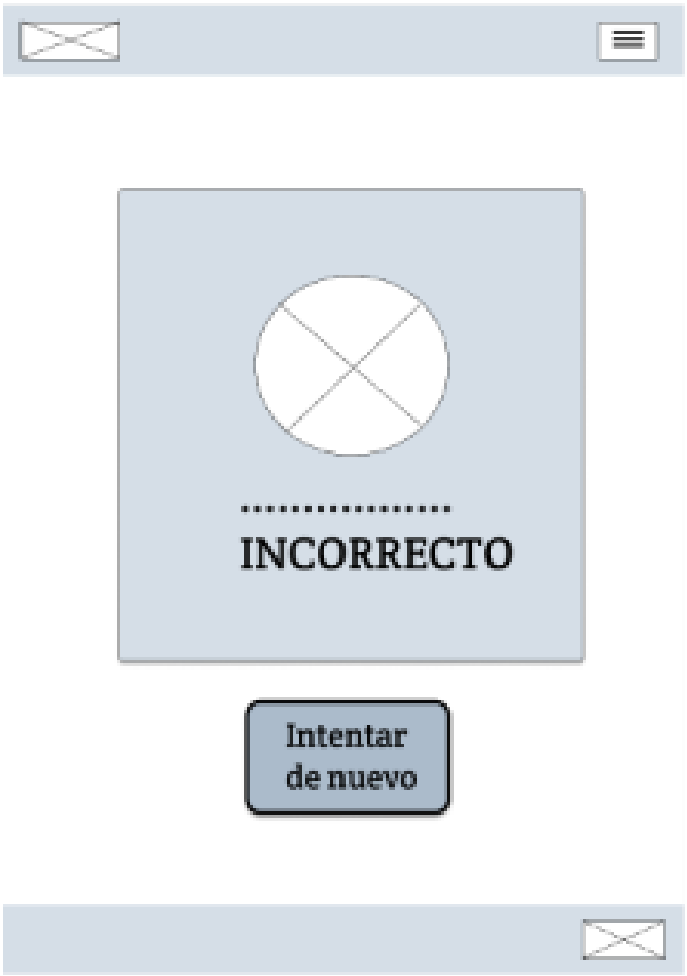


Figura 17: Pantalla de error en el sistema.

3.2. Bocetos Base de Datos

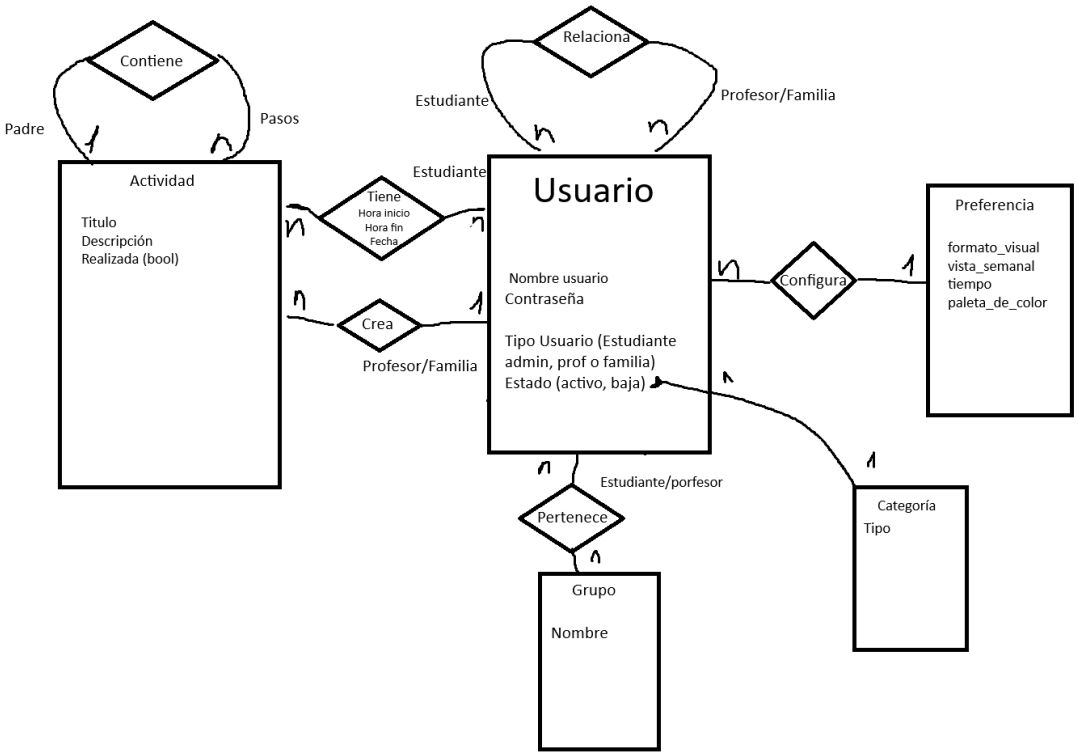


Figura 18: Boceto de la Base de Datos.

4 Manual de Calidad

4.1. Objeto y Alcance del Manual de Calidad

El presente Manual de Calidad establece los principios, metodologías, métricas y controles que garantizan la calidad del producto final y de la documentación generada durante el desarrollo de la aplicación web **Doble Agenda**.

Este plan se aplica de forma continua e integrada a lo largo del ciclo de vida del proyecto (desarrollado bajo la metodología Scrum) y abarca la validación de requisitos de usabilidad y accesibilidad cognitiva, los estándares de código fuente, la trazabilidad documental en LaTeX y el control del trabajo en equipo según las pautas fijadas en el Manual de Coordinación.

4.2. Criterios y Dimensiones Objetivas de Calidad

Para asegurar que las entregas cumplan con los estándares exigidos por el equipo y el cliente, se definen métricas objetivas de obligado cumplimiento evaluadas en cada iteración:

4.2.1. Dimensiones de Calidad para la Documentación (LaTeX)

Siguiendo los estándares fijados en la Sección 3.6 del Manual de Coordinación, la documentación debe cumplir con los siguientes criterios:

- **Compilación y Formato Corporativo (100 %):** Todos los documentos deben compilar sin errores ni advertencias graves en LaTeX, incorporando la portada oficial, encabezados y pies de página corporativos con el logotipo de DobleM y la paleta de colores institucional (`verdeApagado` y `azulPastelApagado`).
- **Trazabilidad de Requisitos (100 %):** Toda funcionalidad o boceto descrito debe estar explícitamente vinculado a sus códigos de requisitos funcionales (RF), no funcionales (RNF) o de información (RI).
- **Calidad Ortográfica y Terminológica:** Ausencia total de errores ortográficos, garantizando una redacción técnica clara, coherente y precisa en español.

4.2.2. Dimensiones de Calidad para el Código Fuente

Para el software desarrollado en la arquitectura MVC (PHP, Twig, JavaScript, HTML, CSS y MySQL) bajo Docker, se aplican los estándares definidos en la Sección 3.7:

- **Estándares de Nomenclatura (100 %):** Cumplimiento estricto del formato `camelCase` para variables y funciones, `PascalCase` para clases y `UPPER_SNAKE_CASE` para constantes. Los identificadores deben escribirse en inglés y los comentarios explicativos en español.
- **Modularidad y Tamaño de Funciones:** Ninguna función o método del backend o frontend superará las 50 líneas de código para garantizar su mantenibilidad.

- **Reutilización y Duplicación (<5 %):** La duplicación del código fuente se mantendrá por debajo del 5 % mediante la modularización de componentes en Twig y funciones globales en PHP/JavaScript.
- **Integridad de la Rama Principal (`main`):** El código subido a la rama principal del repositorio GitHub debe estar libre de errores sintácticos y permitir el flujo de usuario completo sin fallos en tiempo de ejecución.

4.2.3. Dimensiones de Calidad en Accesibilidad Cognitiva y Usabilidad

Dado que el destinatario principal es el alumnado con discapacidad intelectual del PTVAL:

- **Carga Cognitiva Limitada (RNF1):** La interfaz no superará los 15 elementos interactivos contextuales por pantalla.
- **Contraste y Lectura Fácil (RNF3, RNF9):** Cumplimiento del nivel AA de WCAG 2.1 (ratio mínimo 4.5:1 para texto normal) e integración obligatoria de la iconografía de ARASAAC con fuentes sin serifa de tamaño mínimo 18px.
- **Área Táctil (RNF4):** Los botones y elementos interactivos mantendrán un tamaño mínimo de 44×44 píxeles CSS para asegurar su uso en dispositivos de tipo tableta.

4.3. Procedimientos de Control de Calidad durante el Desarrollo

La supervisión de la calidad no es una etapa final aislada, sino una responsabilidad continua coordinada por la **Gestora de Calidad** (Minerva Cebrián Marín) y respaldada por todo el equipo mediante las siguientes acciones:

1. **Seguimiento del Tablero Jira:** Verificación del estado real de las historias de usuario y tareas. Ninguna tarea se dará por "Finalizada"si no ha superado las comprobaciones de código/documentación correspondientes.
2. **Revisiones Cruzadas (*Peer Review*):** Antes de fusionar solicitudes de extracción (*pull requests*) en GitHub o dar por cerrado un capítulo del documento, se realizará la revisión previa del código o texto por parte de un integrante diferente al autor.
3. **Pruebas de Accesibilidad y Emulación:** Validación periódica del diseño adaptativo y táctil mediante herramientas de emulación de dispositivos (Chromium/Safari Device Mode) y verificación de contraste.
4. **Gestión y Corrección de Incidencias:** Todo fallo, inconsistencia tipográfica o desviación en los requisitos detectada se registrará formalmente como una incidencia en Jira, asignándole una prioridad para su resolución inmediata dentro de la iteración en curso.

4.4. Medición del Desempeño y Control de Calidad Interno

Para asegurar la cohesión del equipo y el cumplimiento de los compromisos adquiridos en el Plan de Dirección, la calidad del trabajo colectivo se auditará bajo los siguientes indicadores:

- **Tasa de Cumplimiento de Tareas ($\geq 85\,\%$):** Finalización dentro de los plazos estipulados en cada sprint de la planificación.
- **Trazabilidad del Esfuerzo:** Registro individual de horas dedicadas en la herramienta común con un margen de desviación máximo del $15\,\%$ respecto a lo estimado.
- **Gestión de Estados:** Modificación en tiempo real del estado, fecha de inicio y finalización de cada tarea asignada en Jira.

5 Manual de Despliegue, Arquitectura y Especificación de la API

El presente documento constituye la guía técnica oficial para el despliegue del entorno local, la organización del repositorio y la integración entre los subsistemas de *Front-end* y *Back-end*.

5.1. Preparación del Entorno y Despliegue con Docker

El proyecto utiliza contenedores mediante **Docker Compose** para estandarizar el entorno de desarrollo. Siga las instrucciones correspondientes a su estado inicial.

5.1.1. Inicialización por primera vez (Instalación limpia)

Si nunca se ha compilado el entorno ni realizado la descarga del repositorio, ejecute en la terminal desde la carpeta raíz `app/`:

```
cd app
docker compose up
```

5.1.2. Recompilación del entorno (Entorno previamente montado)

Si ya contaba con el contenedor y se han integrado cambios en la configuración base, fuerce la reconstrucción sin uso de caché:

```
docker compose down
docker compose build --no-cache
docker compose up -d
```

5.1.3. Instalación del motor de plantillas Twig

Una vez que el servidor esté en ejecución, instale las dependencias necesarias ejecutando Composer dentro del contenedor:

```
docker compose exec webserver composer require "twig/twig:^3.0"
```

Importante (Front-end): Para poder renderizar plantillas de Twig correctamente, es **obligatorio** incluir el autoloader de Composer al inicio de cualquier archivo PHP destinado a renderizado:

```
require_once __DIR__ . '/vendor/autoload.php';
```

5.1.4. Despliegue e Importación de la Base de Datos

Para mantener sincronizado el esquema de la base de datos con los últimos cambios de *Back-end*, siga el siguiente procedimiento:

1. Asegúrese de que el servidor está activo y acceda a **phpMyAdmin** en su navegador mediante `http://localhost/phpmyadmin`.
2. En el panel izquierdo, busque la base de datos `agenda`.
3. **Si ya existe:** Selecciónela, diríjase a la pestaña *Operaciones* y ejecute la opción **Eliminar la base de datos (DROP)**.
4. **Creación de BD limpia:** Haga clic en *Nueva* en el menú izquierdo, introduzca el nombre `agenda` y pulse *Crear*.
5. **Importación del Backup:** Seleccione la base de datos `agenda`, acuda a la pestaña superior *Importar*, cargue el archivo situado en:

   `app/BD Backup/agenda.sql`

   y confirme la operación con el botón *Importar*.

5.2. Arquitectura y Estructura del Proyecto

Toda la aplicación se encuentra encapsulada dentro del directorio `app/`. A continuación se especifica la responsabilidad de cada subdirectorio y su equipo encargado:

Directorio / Archivo	Responsable	Descripción y Propósito
`app/api/`	Back-end	Endpoints PHP invocados vía JavaScript/Fetch para responder datos de la BD en formato JSON.
`app/assets/`	Compartido	Recursos globales (hojas de estilo estáticas base, favicon, imágenes globales).
`app/BD Backup/`	Back-end	Contiene los scripts SQL (`agenda.sql`) con el estado más reciente de la BD.
`app/css/`	Front-end	Estilos CSS específicos de la aplicación web.
`app/js/`	Front-end	Archivos JavaScript con la lógica de interactividad, animaciones y peticiones asíncronas (*Fetch*).
`app/services/`	Back-end	Servicios del sistema (gestores de sesión, controladores de conexión a la BD).
`app/templates/`	Front-end	Plantillas Twig y vistas parciales HTML (headers, footers, componentes visuales).
`app/utils/`	Back-end	Funciones auxiliares y glosario de utilidades para consultas a la base de datos.
`app/vendor/`	Sistema	Librerías y autoloader gestionados por Composer (**no modificar**).
`app/renders/`	Front-end	*(Opcional)* Directorio recomendado para organizar los controladores PHP de renderizado de vistas.

Cuadro 2: Organización del directorio principal de la aplicación.

Archivos del Entorno Base (No modificar) Los archivos `index.php`, `phpinfo.php`, `test_db.php`, `test_db_pdo.php`, `composer.json` y `composer.lock` pertenecen a la infraestructura LAMP y gestores de paquetes. No deben editarse salvo requerimiento expreso del equipo.

5.3. Especificación de la API HTTP / JSON

El flujo de información en la plataforma sigue el patrón cliente-servidor asíncrono. Un archivo de renderizado invoca la plantilla Twig; esta carga un script JS que consulta la API pública enviando y recibiendo estructuras JSON.

5.3.1. Estándar de Respuesta JSON

Todos los endpoints ubicados en `app/api/` devuelven una respuesta con la siguiente estructura base:

```
{
"success": true,   // Booleano: indica si la operación fue
exitosa
çode": 0,        // Entero: código de estado/error específico
"message": "..."  // String: descripción legible del resultado
o fallo
}
```

5.3.2. Catálogo de Endpoints Disponibles

1. `crear_estudiante.php`
   - **Función:** Registra un nuevo perfil de estudiante.
   - **Parámetros GET/POST:** `name` (Nombre de usuario), `pass` (Contraseña).
   - **Códigos posibles:** `0` (Éxito), `100` (Falta parámetro), `200` (Usuario ya existe).
2. `crear_profesor.php`
   - **Función:** Registra un nuevo perfil de docente.
   - **Parámetros GET/POST:** `name` (Nombre de usuario), `pass` (Contraseña).
   - **Códigos posibles:** `0`, `100`, `200`.
3. `crear_familiar.php`
   - **Función:** Registra un perfil para tutores o familiares.
   - **Parámetros GET/POST:** `name` (Nombre de usuario), `pass` (Contraseña).
   - **Códigos posibles:** `0`, `100`, `200`.
4. `crear_pertenece.php`
   - **Función:** Asigna la pertenencia de un usuario a un grupo concreto.
   - **Parámetros GET/POST:** `user` (Usuario), `group` (Grupo destino).
   - **Códigos posibles:** `0`, `100`, `201` (Asignación previa), `300`, `301` (Grupo no existe), `400`.

5. `crear_asociacion.php`
   - **Función:** Asocia a un estudiante con su responsable legal o docente.
   - **Parámetros GET/POST:** `student` (Estudiante), `responsible` (Profesor/Familiar).
   - **Códigos posibles:** `0`, `100`, `202` (Asociación existente), `300`, `400`.
6. `consultar_responsable_asociado_estudiante.php`
   - **Función:** Verifica la vinculación de responsabilidad entre un estudiante y un tutor/docente.
   - **Parámetros GET/POST:** `student`, `responsible`.
   - **Códigos posibles:** `0` (Asociado), `1` (No asociado / `success: false`), `100`.
7. `login_estudiante.php`
   - **Función:** Valida credenciales e inicia sesión para la cuenta de estudiante.
   - **Parámetros GET/POST:** `name`, `pass`.
   - **Códigos posibles:** `0`, `100`, `203` (Password incorrecta), `300`, `400`, `401` (Cuenta inactiva).
8. `consultar_agenda_estudiante.php`
   - **Función:** Recupera la agenda de actividades personalizada del alumno en sesión.
   - **Parámetros Opcionales:** `view` (`'daily'` por defecto o `'weekly'`), `date` (`YYYY-MM-DD`), `startDate`, `endDate`.
   - **Códigos posibles:** `0` (Devuelve array de tareas o array vacío si no hay datos), `400` (Sesión inválida/no autorizada).
9. `consultar_actividad_estudiante.php`
   - **Función:** Extrae la información detallada de una tarea (estado actual y desglose de subtareas).
   - **Parámetros GET/POST:** `id_actividad` (Identificador único).
   - **Códigos posibles:** `0`, `100`, `300`, `400`.

5.3.3. Diccionario de Códigos de Respuesta

Código	Estado	Descripción e Interpretación
0	Éxito	Operación realizada correctamente.
1	Éxito Evaluatorio	Consulta completada pero la condición resulta falsa (`success: false`).
100	Error Parámetros	Falta una o varias variables requeridas para procesar la petición.
200	Error Duplicado	El nombre de usuario especificado ya se encuentra registrado.
201	Error Duplicado	La relación de pertenencia a grupo ya existe.
202	Error Duplicado	La asociación entre estudiante y responsable ya está registrada.
203	Error Credenciales	La contraseña proporcionada no es válida.
300	No Encontrado	El recurso solicitado (usuario, actividad, etc.) no existe en el sistema.
301	No Encontrado	El grupo especificado no existe.
400	No Autorizado	Permisos insuficientes o ausencia de sesión activa.
401	Cuenta Bloqueada	El usuario se encuentra dado de baja o inactivo.

Cuadro 3: Codificación estándar de respuestas de la API.